26-03-01 펌웨어 개발 의뢰서

26-03-01 펌웨어 개발 의뢰서

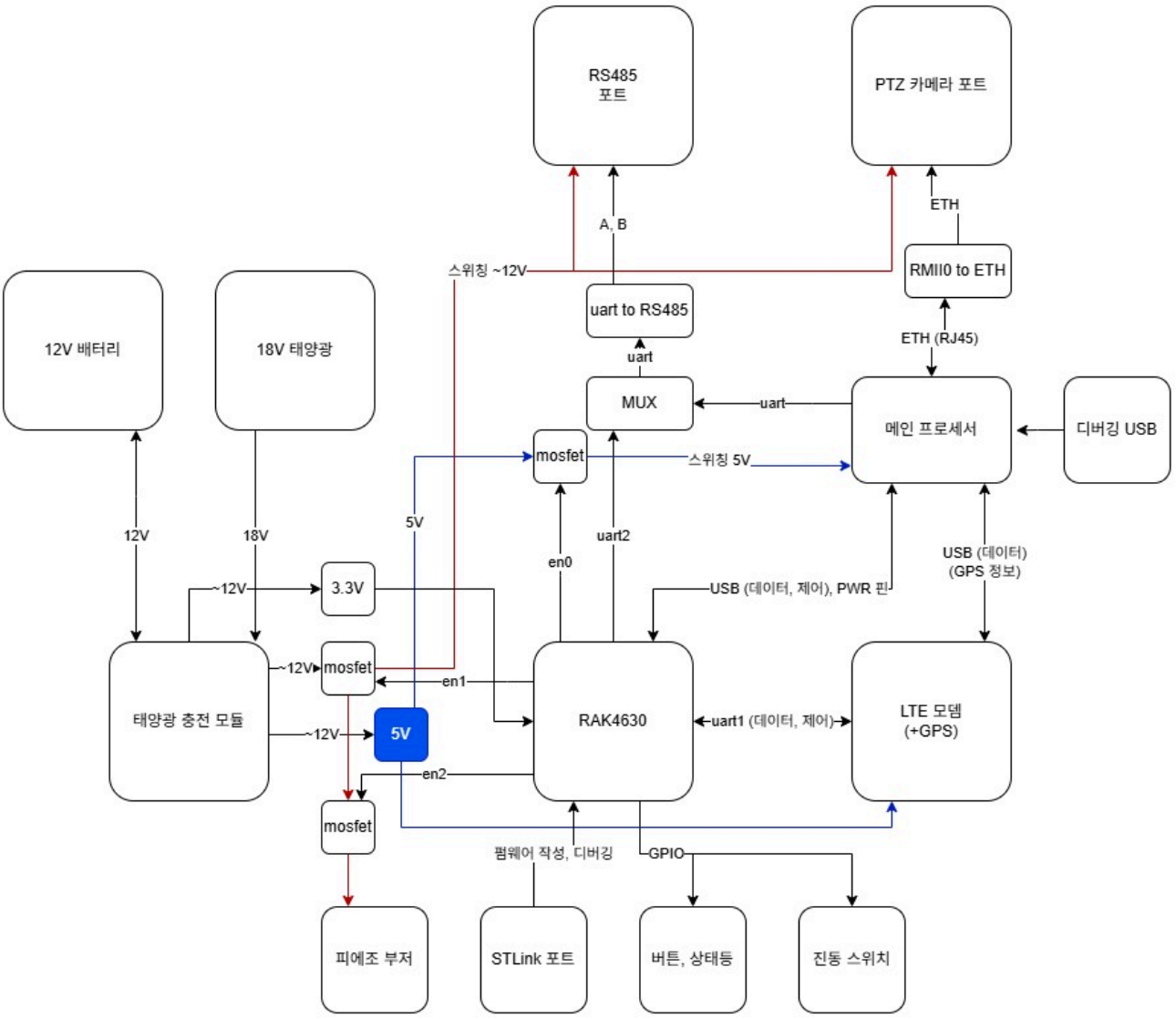
created at: 2026-03-01 14:42

요약

리비타 타워

하드웨어 구성

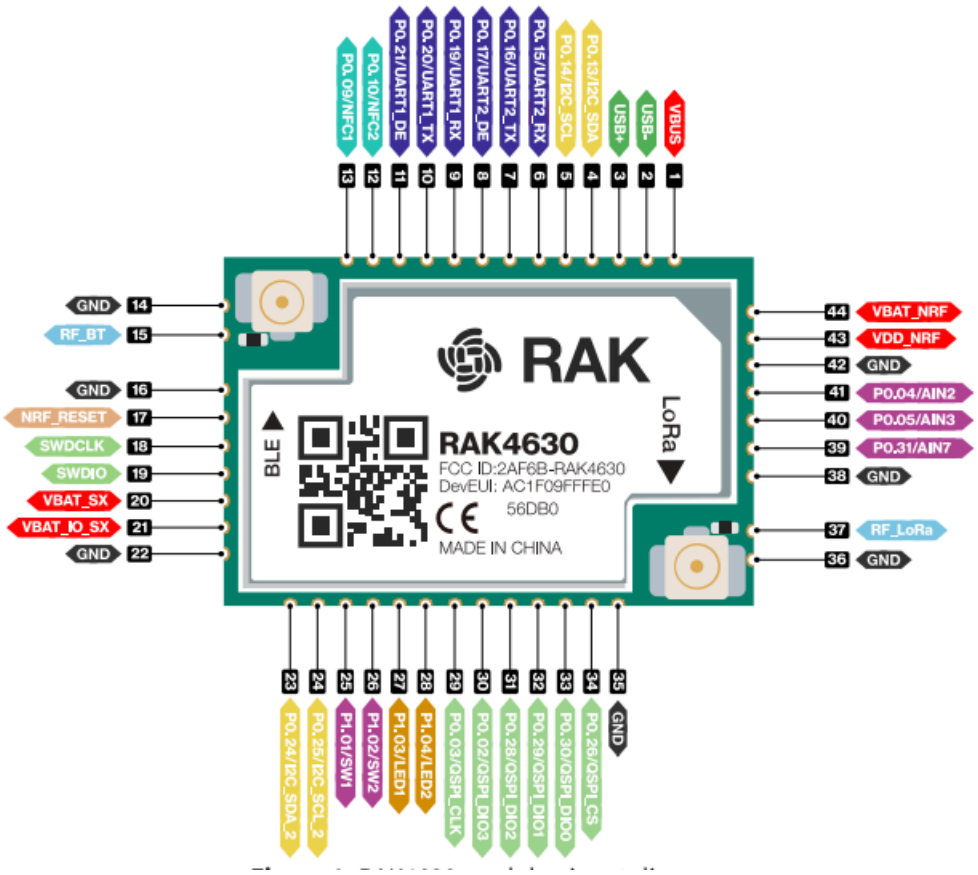
개요



부품 분류	부품, 모듈 명	설명
연산부	RAK4630	1. 핵심 MCU 2. 3.3V 상시 동작 3. BLE, LoRa 안테나 연결 4. 상시 light sleep 또는 deep sleep 5. 외부 flash 연결 6. GPIO 확장기 연결

부품 분류	부품, 모듈 명	설명
		7. LTE와 UART 연결 8. Core USB 연결
	MCP23017-E/SS	1. I2C로 GPIO 확장 2. 3.3V 상시 동작 3. 인터럽트 핀 제공 4. RST 제공
	MX25R1635FZUIL0	1. 플래시 메모리 2. OTA, 속성값 저장용 3. OTA 이외 sleep 모드
LTE 통신부	RM76-4	1. 5V 상시 동작 2. RAK4630과 UART 통신 인터럽트 제공 3. Core와 USB 연결
비디오 처리부	Core3506	1. 5V 제어 동작 2. RAK4630에게서 제어권 양도 받음 3. USB, UART, ETH로 거의 모든 주변 기기와 통신 가능 4. LTE와 USB 연결
전원 제어부	SOLAR_CHARGER_4S	1. 태양광 입력 (직결) 2. 배터리 관리 3. VBAT 출력
	5V-step-down	1. 상시 전원 공급
	3.3V-step-down	1. 상시 전원 공급
센서 연결부	RS485 to uart (3.3V)	1. UART ↔ RS485 2. RE#, DE로 송신, 수신 제어 3. 3.3V 상시 동작 (RE#, DE로 shutdown) 4. 필요 포트 UART x 1 GPIO x 2 (#RE, DE)
	UART_MUX	1. UART, RE#, DE를 MUX로 제어 2. MUX_SEL, MUX_EN#로 전원 및 MUX 제어
외부 전원 스위치	5264 4핀	1. LED+, NC, C, LED- 연결 C = 3.3V LED+ = 아래 스위칭 LED- = GND NO = GPIO 2. 버튼을 누르면 펄스가 MCU로 가서 신호를 줌 3. 필요 포트 GPIO x 1
도난 방지부	SW-18010P (진동 스위치)	1. 진동이 없을 때 off 2. 진동이 있으면 ON 3. 3.3V 상시 전원 사용 4. 필요 포트 GPIO x 1 (강한 전류에 안타게 저항 필요)
	부저 + mosfet	1. EN되어 12V들어오면 0.72W 소리 발생
펌웨어 작성부	STLINK 연결부	1. SWDIO, SWCLK, RST, GND
	DEBUG 부	1. UART TX, RX, GND

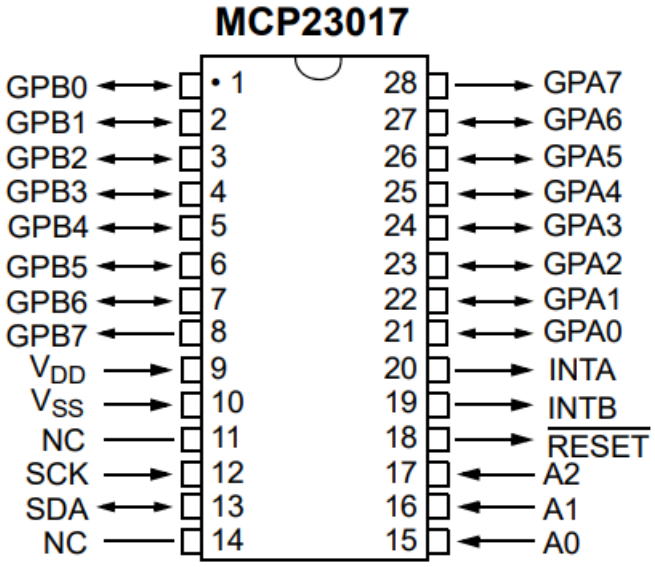
RAK4630 결선



핀 그룹	핀 (NET이름)	핀 번호	특이 사항
STLINK	SWDCLK	18	
	SWDIO	19	
DEBUG	RAK_UART1_RX	6	(RS485와 공유)
	RAK_UART1_TX	7	(RS485와 공유)
FLASH	QSPI_CS	34	
	QSPI_DIO0	33	
	QSPI_DIO1	32	
	QSPI_DIO2	31	
	QSPI_DIO3	30	
	QSPI_SCLK	29	
RS485 UART	RAK_UART1_RX	9	UART_MUX에 연결
	RAK_UART1_TX	10	UART_MUX에 연결
	RAK_RE#	11	UART_MUX에 연결
	RAK_DE	8	UART_MUX에 연결
LTE	RAK_UART2_RX	6	
	RAK_UART2_TX	7	
	LTE_DTR	13	인터럽트 기반 통신
	LTE_RI	12	인터럽트
GPIO 확장	I2C_SCL	12	
	I2C_SDA	13	
SBC 통신	MCU_USB_D_N	2	
	MCU_USB_D_P	3	

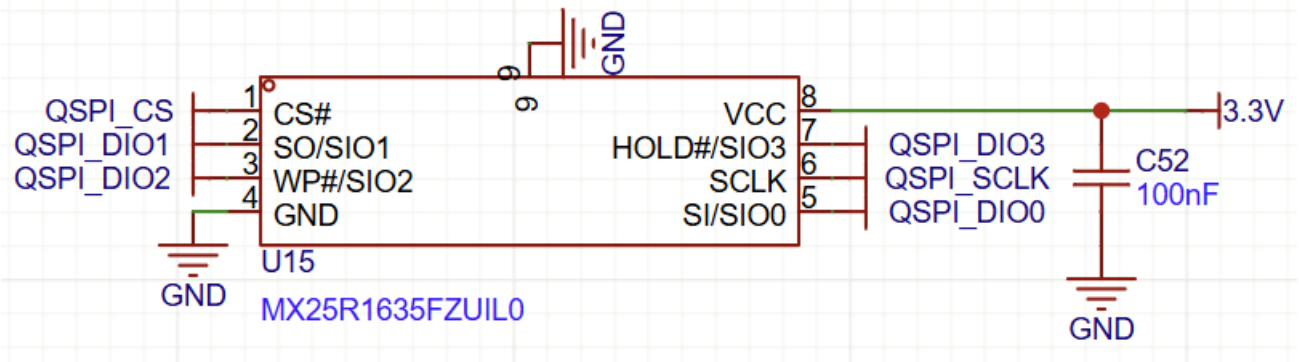
핀 그룹	핀 (NET이름)	핀 번호	특이 사항
전원	BAT_AIN	39	1M/220K 분배 전압
RESET	RST#	17	470K 풀업

MCP23017-E/SS

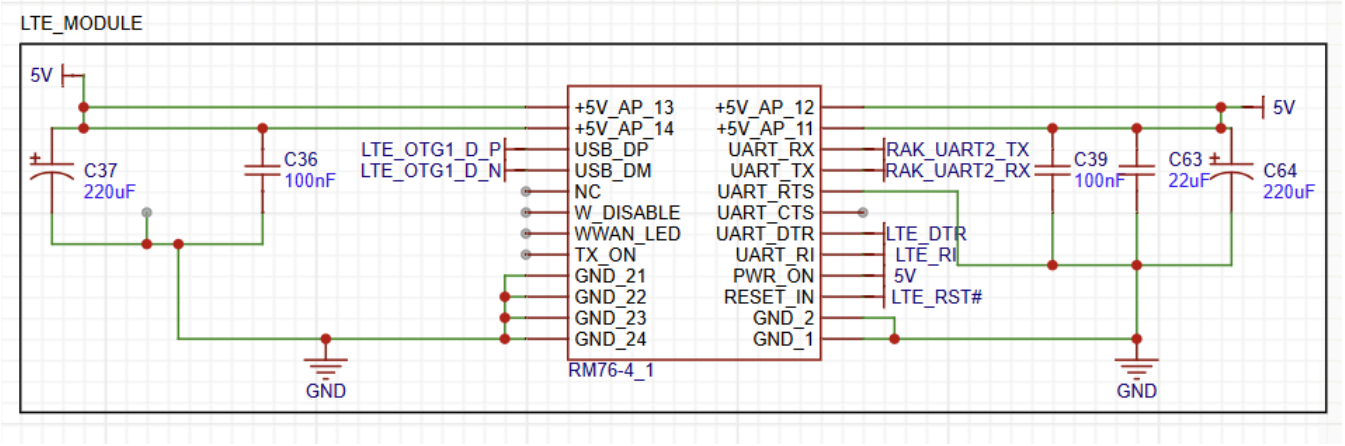


핀 그룹	핀 (NET이름)	핀 번호	특이 사항
RAK4630과 통신	I2C_SDA	12	
	I2C_SCL	13	
	RST#	18	470K 풀업
CAM 전원 관리	CAM_EN	28	CAM_VDD 전원 관리 470K 풀다운
5V 전원 관리	5V_EN	27	SBC 전원 관리 470K 풀다운
3.3V 전원 관리	3.3V_EN	26	ETH 전원 관리 470K 풀다운
RS485 전원 관리	12-14V_EN	23	RS485 전원 관리 470K 풀다운
LED	LED_EN	7	10K 풀다운
부저	BUZZER_EN	2	10K 풀다운
SBC 리셋	SBC_RST#	1	100K 풀업
LTE 리셋	LTE_RST#	21	
UART_MUX	MUX_EN#	25	100K 풀업
	MUX_SEL	24	

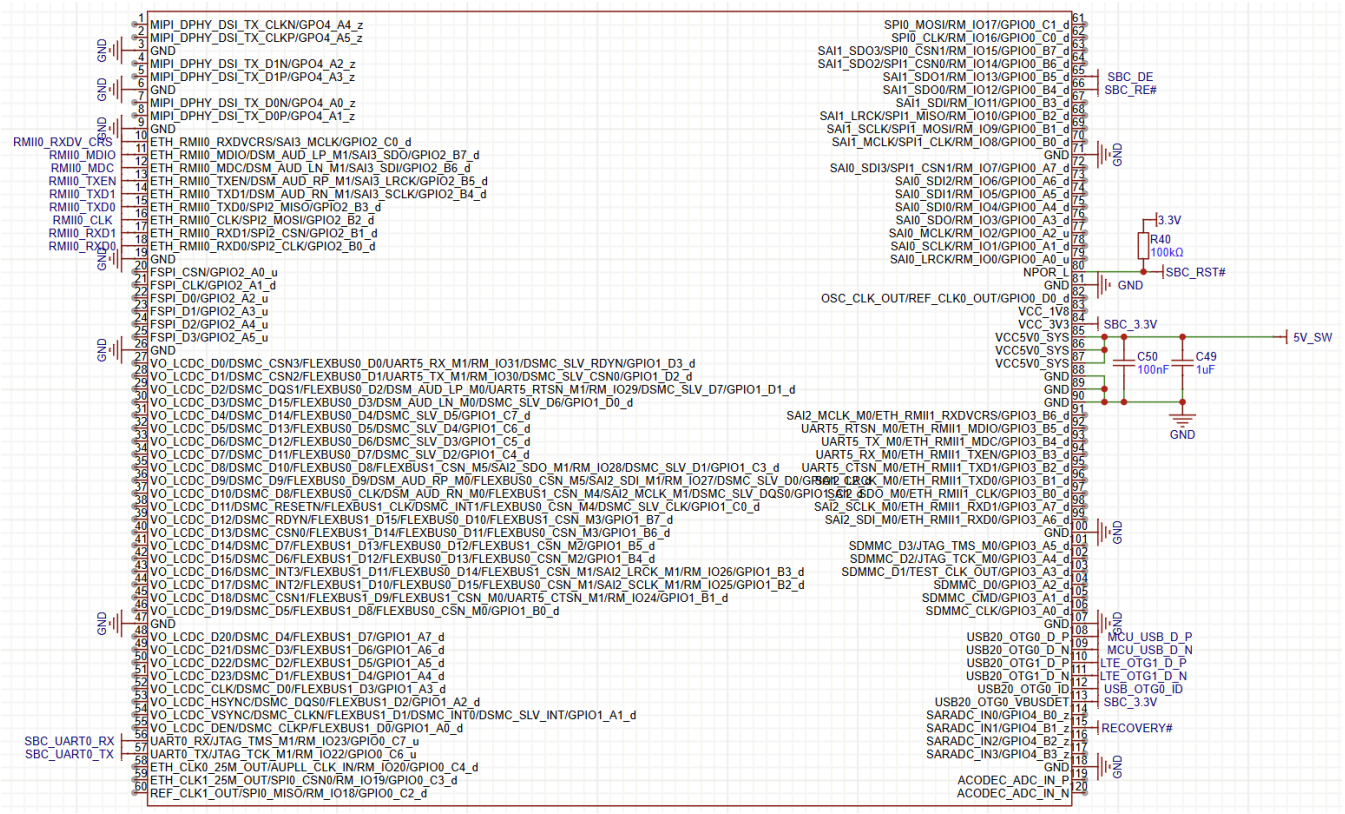
MX25R1635FZUIL0 (FLASH)



LTE 모듈



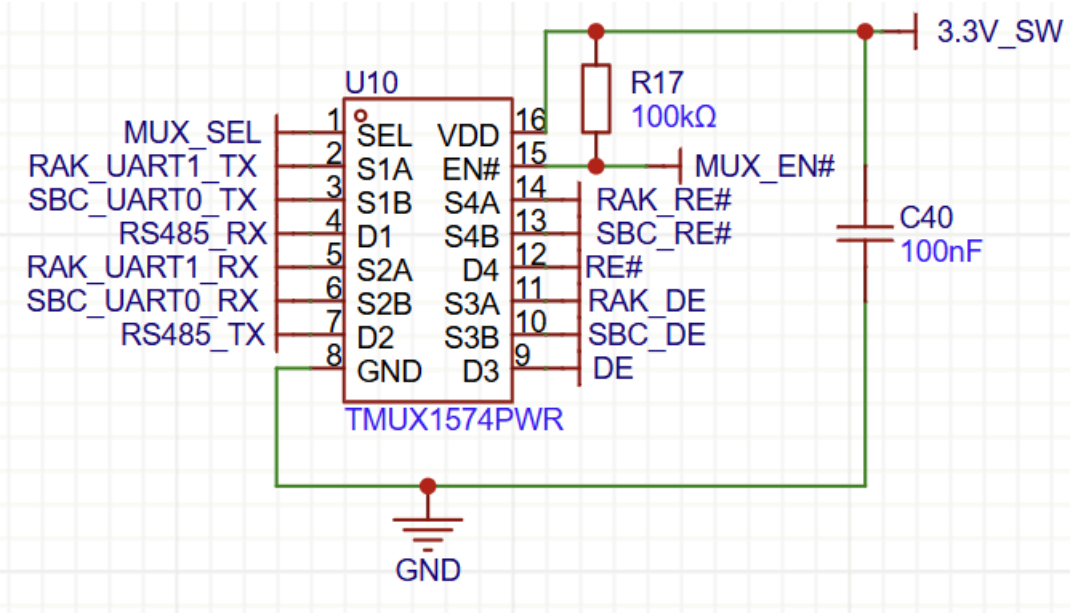
SBC



핀 그룹	핀 (NET이름)	핀 번호	특이 사항
RAK4630과 통신	MCU_USB_D_P	108	
	MCU_USB_D_N	109	
	SBC_RST#	80	100K 풀업

핀 그룹	핀 (NET이름)	핀 번호	특이 사항
LTE와 통신	LTE_OTG1_D_P	110	
	LTE_OTG1_D_N	111	
USB OTG 관리	USB_OTG0_ID	112	test point로 high/low 관리
펌웨어 업데이트	RECOVERY#	115	버튼으로 관리
RS485 통신	SBC_UART0_RX	56	
	SBC_UART0_TX	57	
	SBC_DE	65	
	SBC_RE#	66	

TMUX1574PWR (UART_MUX)

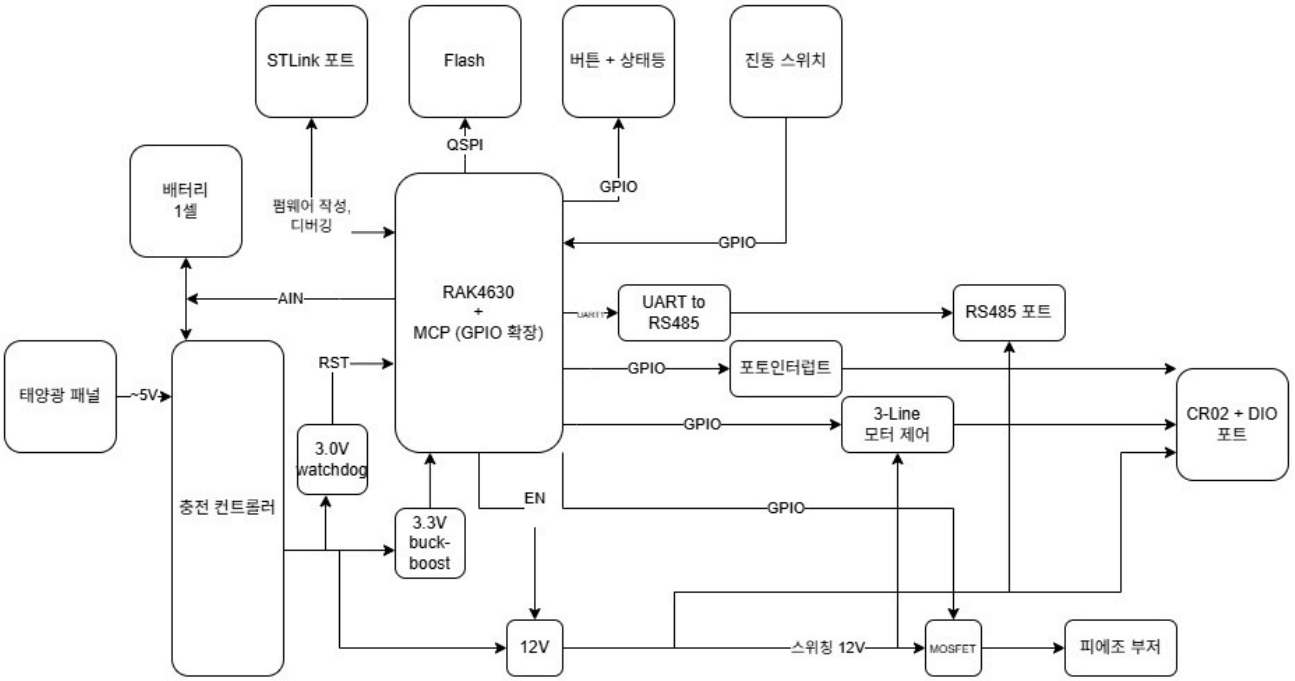


핀 그룹	핀 (NET이름)	핀 번호	특이 사항
MUX 제어	MUX_EN#	15	100K 풀업
	MUX_SEL	1	
SBC 방향 입력	SBC_UART0_TX		
	SBC_UART0_RX		
	SBC_RE#		
	SBC_DE		
RAK 방향 입력	RAK_UART1_TX		
	RAK_UART1_RX		
	RAK_RE#		
	RAK_DE		
출력	RS485_RX		(TX와 연결)
	RS485_TX		(RX와 연결)
	RE#		
	DE		

리비타 링크

하드웨어 구성

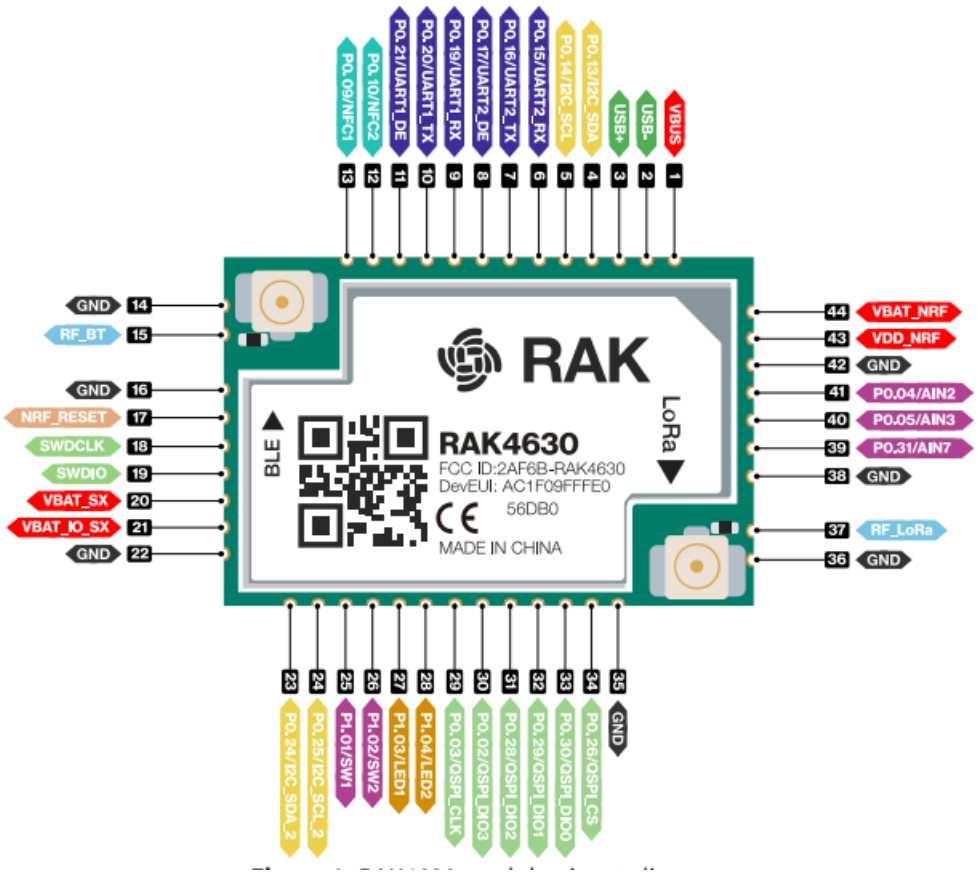
개요



부품 분류	부품, 모듈 명	설명
연산부	RAK4630	1. 핵심 MCU 2. 3.3V 상시 동작 3. 3.0V watchdog 연결 (RST 제어) 4. BLE, LoRa 안테나 연결 5. 상시 light sleep 또는 deep sleep 6. 외부 flash 연결 7. GPIO 확장기 연결
	MCP23017-E/SS	1. I2C로 GPIO 확장 2. 3.3V 상시 동작 3. 인터럽트 핀 제공 4. RST 제공
	MX25R1635FZUIL0	1. 플래시 메모리 2. OTA, 속성값 저장용 3. OTA 이외 sleep 모드
전원 제어부	SOLAR_CHARGER_1S (배터리 충전)	1. 태양광 입력 (직결) 2. 배터리 관리 (직결) 3. LOAD_VCC 출력 (2.5~5V) 각 부하에 커페시터 추가 필수 4. 온도 센서는 배터리 근처에 배치! 5. 필요 포트 AIN x 1 (배터리 전압 감지)
	3.3V_buck_boost	1. 연산부 전력 공급 (상시)
	3.0V-watchdog	1. 배터리에 직결 2. 배터리 전압이 3.0V 이하이면 RST 신호 발생 (전력 차단) 3. 연결 포트 RST x 1
	12V-step-up	1. 출력 12V로 승압 2. EN으로 제어

부품 분류	부품, 모듈 명	설명
		3. 필요 포트 GPIO x 1 (EN)
센서 연결부	RS485 to uart (3.3V)	1. UART ↔ RS485 2. RE#, DE로 송신, 수신 제어 3. 3.3V 상시 동작 (RE#, DE로 shutdown) 4. 필요 포트 UART x 1 GPIO x 2 (#RE, DE)
벨브 제어부	3-Line-motor-control x 2	1. GPIO → 3선 전력 2. 12V 스위칭 전력 연결 3. 필요 포트 GPIO x 3 (EN_A, EN_B, EN_P2)
유량계 부	12V DIO to 3.3V DIO x 2	1. 12V dio → 3.3V dio 2. 필요 포트 GPIO x 1 (MCU에 직결)
외부 전원 스위치	5264 4핀	1. LED+, NC, C, LED- 연결 C = 3.3V LED+ = 아래 스위칭 LED- = GND NO = GPIO 2. 버튼을 누르면 펄스가 MCU로 가서 신호를 줌 3. 필요 포트 GPIO x 1
도난 방지부	SW-18010P (진동 스위치)	1. 진동이 없을 때 off 2. 진동이 있으면 ON 3. 3.3V 상시 전원 사용 4. 필요 포트 GPIO x 1 (강한 전류에 안타게 저항 필요)
	부저 + mosfet	1. EN되어 12V들어오면 0.72W 소리 발생
펌웨어 작성부	STLINK 연결부	1. SWDIO, SWCLK, RST, GND
	DEBUG 부	1. UART TX, RX, GND

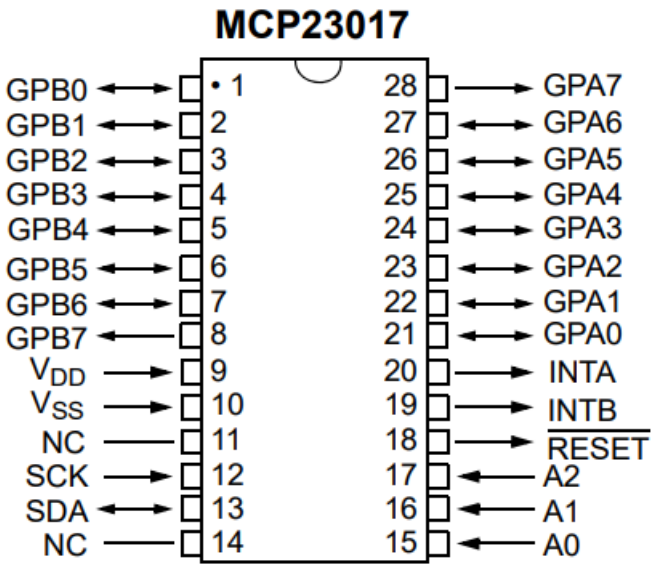
RAK4630 결선



핀 그룹	핀 (NET이름)	핀 번호	특이 사항
STLINK	SWDCLK	18	
	SWDIO	19	
DEBUG	UART2_RX	6	
	UART2_TX	7	
FLASH	QSPI_CS	34	
	QSPI_DIO0	33	
	QSPI_DIO1	32	
	QSPI_DIO2	31	
	QSPI_DIO3	30	
	QSPI_SCLK	29	
RS485 UART	UART1_RX	9	10K 풀업
	UART1_TX	10	10K 풀업
	RE#	27	10K 풀업
	DE	28	10K 풀다운
DIO X	DIO_X	12	10K 풀업 다운 엣지
DIO Y	DIO_Y	13	10K 풀업 다운 엣지
GPIO 확장	I2C_SDA	4	4.7K 풀업
	I2C_SCL	5	4.7K 풀업
	INTA	25	
	INTB	26	

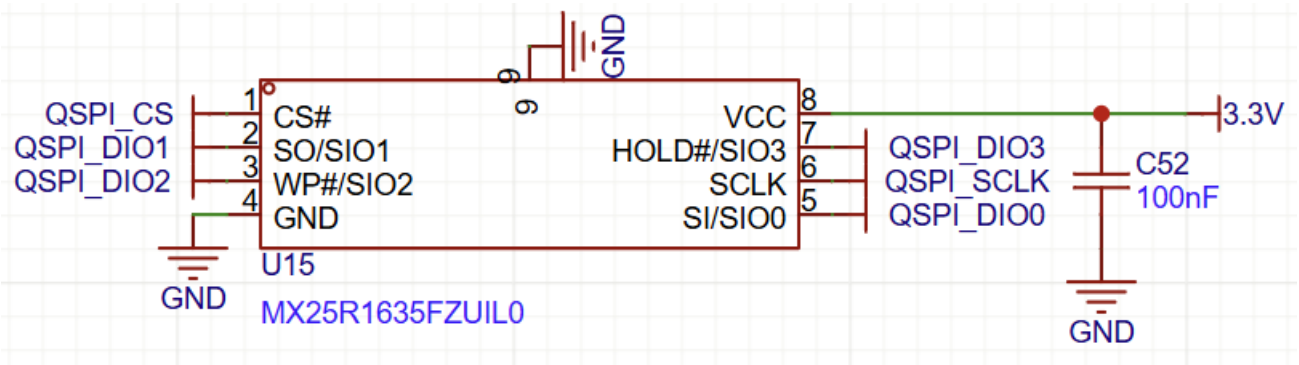
핀 그룹	핀 (NET이름)	핀 번호	특이 사항
진동 센서	VIB_SENSE	11	SW 풀업 유지 필요 다운 엣지
배터리 전압 측정	BAT_AIN	39	인산철 1S 1M/1M 분배 전압
RST	RST#	17	STLINK 3.0V watchdog이 trigger

MCP23017-E/SS



핀 그룹	핀(NET 이름)	핀 번호	특이 사항
RAK4630과 통신	I2C_SDA	12	4.7K 풀업
	I2C_SCL	13	4.7K 풀업
	INTA	20	
	INTB	19	
	RST#	18	STLINK 3.0V watchdog이 trigger
X 그룹 CR02 제어	X_EN_A	24	470K 풀다운
	X_EN_B	23	470K 풀다운
	X_EN_P2	22	10K 풀다운
Y 그룹 CR02 제어	Y_EN_A	27	470K 풀다운
	Y_EN_B	26	470K 풀다운
	Y_EN_P2	25	10K 풀다운
12V 전원 제어	12V_EN	28	100K 풀다운
외부 버튼	BTN	21	10K 풀다운
	LED_EN	8	10K 풀다운

MX25R1635FZUIL0 (FLASH)



공통 펌웨어 구성

전력 인프라 없이 야외에서 동작하는 장비인 만큼 저전력, 안정적으로 동작하는 것이 핵심임.
핵심적으로 요구되는 성능은 다음과 같음.

- 동적 설정 기능
  현장별로 매우 다양한 상황에 맞닿은 만큼 확장성이 매우 중요한 요소임.
  그러한 상황마다 펌웨어를 업데이트하는 것이 아닌 config 데이터 기반 동적 설정 및 데이터 구동형 제어가 가능해야 함.
  - 통합 config 파일 설정
    한 기기의 config는 한 파일로 관리되어 추적 및 설정이 가능해야 함.
    config는 재부팅 후에도 유지되어야 함.
  - 동적 스케줄 관리
    고정된 루프가 아닌 config를 통해 작업 주기, 시작 시간, 활성화 여부를 실시간으로 수정 및 적용 가능해야 함.
    스케줄은 재부팅 후에도 유지되어야 함.
  - 가상 장비 매핑
    하드웨어 핀-드라이버는 고정되지 않고
    드라이버 또한 통합 config 내에 내장되어야 함.
  - 플러그 앤 플레이 대응
    장비 연결 시 수동으로 "기기 추가" 명령을 통해 해당 드라이버 다운 또는 활성화가 가능해야 함.
- 저전력 대기
  태양광-배터리 기반으로 동작하며 대부분의 시간을 IDLE 상태로 보내게 됨.
  이 시기에 전력을 절약하는 것이 전체 성능을 좌지우지할 만큼 핵심임.
  스케줄 또는 외부 인터럽트를 통해서 wake 하여 동작 수행.
  또한 연결되는 외부 기기들도 대기 전력 관리를 위해 mosfet을 통한 전력 스위칭 구조로 되어 있음.
  MCU가 외부 기기를 활용할 때만 전력을 공급해야 함.
  그외 시간은 전부 전력 차단 및 deep, light sleep
  - 스케줄 기반 프레임워크
  - 인터럽트 기반 제어
  - 외부 전원 제어 로직
- 원격 가용성 및 유지보수
  야외에 설치된 장비 특성상 A/S를 하기가 매우 까다로움.
  이를 위해 원격에서 고객의 제품을 진단, 설정하거나 OTA를 하는 기능이 필요합니다.
  - 자체 기능 복구
    소프트웨어나 외부 기기의 오작동이 발생해도 특정 시간 이내에 리셋을 통해 정상 상태로 복구되어야 함.
  - Dual-bank OTA (LoRa, LTE, BLE)
    외부 flash 이용해 OTA 실패 시 기존 버전으로 Rollback
  - 원격 로그 분석
    원한다면 상세 로그를 분석해 현장을 방문하지 않고도 문제 원인을 파악하고 안내할 수 있어야 함.

    - 물리적 강제 복구 수단
      소프트웨어적으로 완전 꼬여서 에러가 발생할 시 외부 버튼으로 공장 초기화로 복구되는 기능이 필요함.
- 통신 및 보안
  하드웨어적 통신 보안은 갖추어져 있으나 소프트웨어적 통신 보안이 갖추어져야 함.
  인증받지 않은 외부 기기가 통신망에 접속해서는 안됨.
  또한 통신을 연결할 때 서버와 시간이나 상태를 동기화하여 정상적인 동작을 보장할 수 있어야 함.
    - LTE, LoRa 통신 보안 (인증서)
    - 통신 인증
    - OTA 검증
    - 시간 동기화
    - 설정 일치 검증
- 통신, 동작 fallback
  농업 장비는 성능보다는 안정성이 핵심 요소임.
  동작을 안정적으로 수행해야 하며, 수행하지 못했다면 수행하지 못함을 확실히 알아야 함
    - 통신 fallback
    - 명령 fallback
      세부적인 ack를 두어 송신측에선 명령 수신을 확인해야 함.
      동일한 명령이 2번 수행되는 사항이 발생해서는 안됨.
      명령을 수행할 수 없는 상태(배터리 부족, 연결 끊김)라면 수행하지 말아야 함.
      외부 기기 이상일 경우 재부팅을 하는 등의 fallback을 수행해야 함.

세부 요구 성능

동적 설정 기능

기능명	요구 사항	목표 성능
통합 config 파일 설정	1. 한 기기의 config는 한 파일로 관리되어 추적 및 설정이 가능해야 함. 2. config는 재부팅 후에도 유지되어야 함.	1. 비휘발성 config 2. 가변 길이 config 가능 3. config 작성 중 리셋이 발생해도 복구 가능해야 함. 4. config 파일의 무결성 검사 가능해야 함.
동적 스케줄 관리	1. 고정된 루프가 아닌 config를 통해 작업 주기, 시작 시간, 활성화 여부를 실시간으로 수정 및 적용 가능해야 함. 2. 스케줄은 재부팅 후에도 유지되어야 함.	1. 부팅 시 config 기반 스케줄 생성 2. config 수정 및 스케줄 업데이트 기능 3. 여러 스케줄이 한 장치를 이용할 때 충돌이 발생해서는 안됨 4. 각 스케줄 성격에 따라 순차 실행 또는 동시 실행 가능해야 함.
가상 장비 매핑	1. 드라이버 또한 통합 config 내에 내장되어야 함. 2. 드라이버는 API를 제공하며 명령은 이 API를 활용하는 형태로 동작함.	1. 센서 드라이버가 config에 내장되야 함. 2. 표준에 해당하는 장치에 대해선 드라이버 교체 수준으로 전부 지원 가능해야 함. 3. 명령은 드라이버의 API만을 활용해야 함.
플러그 앤 플레이 대응	1. 장비 연결시 BLE 또는 서버를 통해 "기기 추가" 명령을 내려 해당 드라이버 다운 또는 활성화가 가능해야 함.	1. 드라이버 다운 명령 지원 2. 드라이버 다운 관련 config 재설정 지원

저전력 대기

기능명	요구 사항	목표 성능
스케줄 기반 프레임워크	1. 스케줄은 프레임워크로 관리되어 추후 코드 유지 보수에 적합해야 함. 2. 스케줄 사이 IDLE 시간에는 무조건 deep, light sleep이 동작해야 함. 3. 모든 동작은 독립된 Task 단위로 관리되어야 함.	1. 스케줄 기반 프레임워크 내장
인터럽트 기반 제어	1. 인터럽트 요소 (스케줄, 통신, 진동 센서, 버튼)에 의해 wakeup하고 지정된 동작을 수행해야 함.	1. 인터럽트 요소에 대응된 이벤트는 인터럽트 관련 동작을 완전히 처리해야 함 2. sleep 전에는 인터럽트 요소 정리 확인 후 sleep을 해야 함.
외부 전원 제어 로직	1. 외부 기기 사용할 때만 전원을 공급해야 함 2. 외부 기기 전원 공급 시 warm-up 딜레이를 주어야함 3. 한번에 여러 기기를 사용시 순간 peak current를 넘지 말아야 함.	1. 전원 넣지 않고 사용하는 오작동이 없어야 함. 2. config에 딜레이가 파라미터로 관리되어야 함 3. 기기별 peak current, 순간 peak current 상한은 파라미터로 관리되어야 함

원격 가용성 및 유지보수

기능명	요구 사항	목표 성능
자체 기능 복구	1. 소프트웨어나 외부 기기의 오작동이 발생해도 특정 시간 이내에 리셋을 통해 정상 상태로 복구되어야 함.	1. N시간 이내 정상 상태 복구 보장 2. 오류 발생시 오류 감지 및 해당 오류 로그 작성 후 송신
OTA	1. LoRa, BLE (휴대폰 앱), LTE를 통해 OTA 진행 2. 외부 flash 이용해 OTA 실패시 기존 버전으로 Rollback	1. LoRa, BLE, LTE를 통한 OTA 확인, 다운로드 동작 구현 2. 외부 flash 이용해 OTA 실패시 기존 버전으로 Rollback하는 기능 구현
원격 로그 분석	1. INFO, DEBUG 등으로 로그 레벨이 분리되어야 함. 2. 로그 레벨이 config로 설정 가능해야 함. 3. 로그가 충분히 상세해서 발생한 오류 분석이 가능해야 함.	1. 로그 레벨 구현 2. config에 로그 수준 설정 구현 3. 오류 분석 가능한 수준의 로그 기록 4. flash에 저장되어 비휘발성 확보
물리적 강제 복구 수단	1. 외부 전원 버튼을 특정한 패턴으로 누를 시 공장 초기화가 동작해야 함.	1. 어떤 상황이든 특정 패턴의 입력을 감지하고 초기화할 수 있어야 함.

통신 및 보안

기능명	요구 사항	목표 성능
통신 fallback	1. 외부 장애로 통신이 두절되었을 시 자동 복구 시도해야함. 2. LoRa에서 메세지가 정확히 전달되었는지 송신 측에서 확인 가능해야 함.	1. 무한 재시도 로직 구현 2. LoRa ACK 구현 3. 중복 메세지 방지
명령 fallback	1. 송신 측에서 명령 송신을 확인할 수 있어야 함. 2. 명령이 중복 실행되서는 안됨. 3. 명령이 수행할 수 없는 상태라면 수행하지 말아야 함. 4. 설정된, 동작 중인 명령에 대해서 취소, 즉시 중단 등도 지원해야 함.	1. LoRa ACK 구현 2. 중복 명령 수행 방지 3. 명령 수행 필수 조건 파라미터 화 4. 명령 삭제, 취소 기능 지원

기능명	요구 사항	목표 성능
		5. 수신 명령 비휘발성 확보 6. 명령 처리 상태 알림 지원

리비타 링크

- 설명
  리비타 링크는 자체 LoRa 단말기에 RS485, DIO 형태로 센서, 제어기가 추가 결합할 수 있는 노지 스마트팜 단말입니다.
  LoRa를 통해 리비타 타워와 통신하며
  RS485로 연결된 센서에서 데이터를 수집하며
  DIO로 밸브, 모터를 제어하며
  데이터, 명령 수행 결과를 LoRa로 송신합니다.
- 통신 관련 역할
  - LoRa로 단말 역할 수행
    LoRa Mesh 오픈소스 사용
  - RS485로 센서 데이터 수집 또는 제어
  - DIO로 모터 벨브 제어 또는 유량계 확인
  - BLE로 사용자의 앱에서 OTA, 등록 수행
- 관측 동작
  - 센서 통한 데이터 수집 및 송신
    RS485 센서와 드라이버를 통해 센서 데이터 수집하여 LoRa로 송신
- 제어 동작
  - 관수, 제어 명령 수행
    LoRa로 명령 수신 및 명령에 따라 RS485, DIO 조절해 명령 수행
    명령 수행 상태, 수행 결과 LoRa 로 송신
- 방범 동작
  - 도난 감지
    진동 센서를 통해 일정량 이상의 진동 발생시 부저음을 울리고 LoRa로 도난 사실 송신
- 관리 동작
  - 전원 버튼
    전원 버튼을 특정 패턴으로 눌러서 기능 활성화
  - BLE 접속
    BLE를 접속해 사용자 앱에서 상태 확인 및 세부 설정 (ex: LTE 연결 없을 시 원격으로 상태 확인)
  - 상태 확인
    전원 버튼 옆의 LED의 점멸 패턴을 통해 기초 상태 표시 (ON, OFF, 페어링 등)

상세 동작 설명

HAL 레벨 동작

HAL 레벨에서는 하드웨어 레벨의 동작을 정의하고 함수화하여 상위 동작이 하드웨어 레벨의 복잡도를 가지지 않게끔 하기 위함입니다.

LoRa 드라이버

- 설명
  리비타 타워와 통신하며, 리비타 링크를 동작시키는 가장 중요한 통신 드라이버입니다.
  이 드라이버의 내용을 바탕으로 리비타 링크는 동작하게 됩니다.
  통신은 LoRa Mesh 중계기를 거치는 것이 아니라면 무조건
  리비타 타워 <-> 리비타 링크의 스타 네트워크만 구성하며 리비타 링크 <-> 리비타 링크 끼리 명령을 주

고 받을 일은 없습니다.
리비타 링크는 인증된 리비타 타워와만 통신을 구성해야 합니다.

- 참고 라이브러리
  https://meshcore.co.uk/
  https://github.com/meshcore-dev/MeshCore
  (Mesh Core)
  라이브러리에서 이미 정의된 동작들
  - LoRa 통신을 위한 RAK4630 제어 드라이버
  - Mesh 망에서 최적 경로 찾는 알고리즘
  - 최초 통신 (Advert)할 때 암호화 인증서 교환
  - 노드 주소 지정
  - 라우팅 테이블 자동 갱신
  - 수신 후 ACK 송신
  - 자동 재전송
  - 패킷 분할 및 재조립
- 필요 동작
  - 초기화
    통신 파라미터, 자체 ID 설정
  - 네트워크 구성
    네트워크를 구성해서 라우팅 경로를 생성합니다.
    이때 통신하는 대상이 리비타 타워임을 인증하는 프로토콜 및 인증이 필수적입니다.
    ex 1) 통신 거리에 리비타 타워 2개가 있어도 유저가 소유한 리비타 타워와만 소통해야 함.
    ex 2) 리비타 장비가 아닌 장비가 해당 네트워크에 들어와서 구성되면 안됨. (다른 소유자의 리비타 장비 또는 MeshCore를 쓰는 다른 장비)
  - 송신
  - 수신
    데이터 수신 후 ACK 송신 및 수신 데이터 표시
  - 절전
    송신 내용 없을 때 sleep 설정

BLE 드라이버

- 설명
  사용자가 근처에서 폰으로 리비타 링크를 설정하고 싶을 때 사용합니다.
  BLE는 상시 가동은 아니며 후술할 전원 관리 드라이버를 통해 BLE Advert, 페어링 상태에 진입할 때만 동작합니다.
- 필요 동작
  - 초기화
  - BLE advert
    BLE advert 상태일 때 advert를 일정 기간 동안 수행
  - BLE 연결
    BLE 페어링 및 연결 수행
  - BLE 통신
    BLE를 통해 리비타 링크의 상태, 설정 옵션을 제공 (특히 OTA)

Flash 드라이버

- 설명
  Flash 드라이버는 OTA 용도로만 사용 예정입니다. (dual-bank)
  따라서 OTA를 제외한 거의 모든 시간동안 power down으로 사용하지 않는 것이 핵심입니다.

- 참고 라이브러리
  https://github.com/adafruit/Adafruit_SPIFlash/tree/910c72df68ad5938292d2163398a2888be85a701
  (MX25R1635F)
- 필요 지원 동작
  - 초기 연결
    부팅되어 Flash 통신 관련 초기화 수행
  - 읽기
  - 쓰기
  - power down
    flash 메모리를 사용하지 않는 대부분의 시간 동안을 위해 절전 모드를 설정합니다.
  - wake up
    sleep 중인 flash를 깨웁니다.

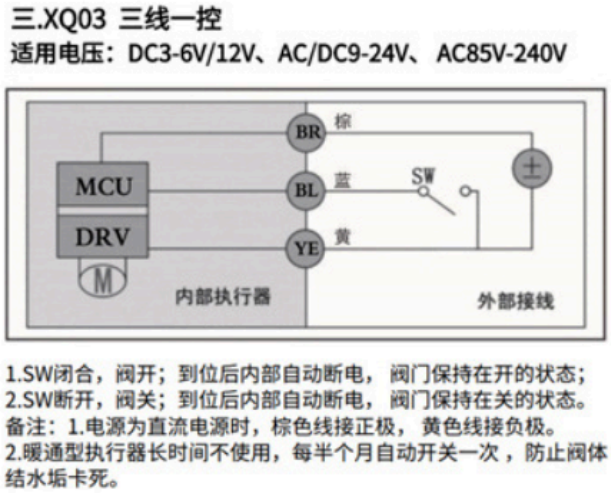
RS485 드라이버

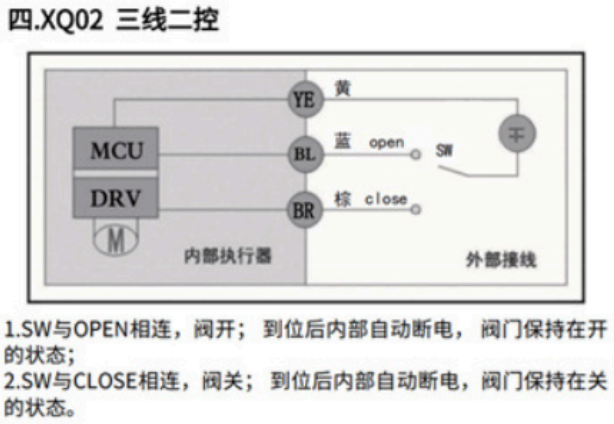
- 설명
  UART, RE#, DE로 MAX3485와 연결되어 통신합니다.
  MODBUS보다 더 낮은 HAL 레벨의 드라이버입니다. (MODBUS는 이 드라이버 위에서 동작 예정입니다)
- 필요 지원 동작
  - 초기화
    GPIO, 통신 세팅등을 활성화합니다.
  - 송신 모드 전환
  - 수신 모드 전환
    기본 상태입니다, 송신이 완료된 후 전환되어야 합니다.
  - 데이터 송신
    반이중 통신을 고려한 충돌 방지 로직이 있어야 합니다.
  - 데이터 수신
    데이터 수신은 항상 송신 직후 발생합니다. 송신 없이 수신이 발생할 가능성은 없습니다. (항상 master가 query)
    응답 타임아웃이 설정되어야 합니다.
  - power down
    MAX3485의 전원을 끕니다. (RE#, DE로 제어 가능)
    또한 UART의 전원마저 꺼져야 합니다.
  - wake up
    UART, MAX3485를 활성화 합니다.

3-Line 모터 제어 드라이버

- 설명
  3선식 모터 제어를 담당합니다.
  사진의 XQ02, XQ03은 각각 CR02, CR03 타입으로 부르고 있습니다.
  리비타 링크에 동일한 회로가 총 2개 있습니다.
  모터 밸브는 솔레노이드와 달리 전원이 내려가도 닫히지 않습니다.

따라서 급작스러운 재부팅 상황 때 이에 대한 fallback이 필요합니다.





- 필요 지원 동작
  - 초기화
  - 설정
    드라이버 세팅을 설정합니다. (CR02, CR03)
  - fallback
    직전 상태를 flash에 기록해, 만약 오류로 전원 재부팅 등이 되었다면 즉시 close 동작을 수행하고 오류를 발송합니다.
  - 개방
    밸브를 개방합니다.
    밸브 번호, 개방 소요 시간이 인자로 전달됩니다.
  - 폐쇄
    밸브를 폐쇄합니다.
    밸브 번호, 폐쇄 소요 시간이 인자로 전달됩니다.

유량계 드라이버

- 설명
  3선식 모터와 동일한 인터페이스로 모터 벨브 직전, 직후에 연결됩니다.
  리비타 링크에 동일한 회로가 총 2개 있습니다.
  이 유량계가 동작하는 동안 소프트웨어적으로 count하는 것이 아닌 background counting (PPI counting)으로 sleep 상태를 유지할 수 있어야 합니다.
- 필요 지원 동작
  - 초기화
  - 설정
    유량계의 설정 (펄스 수 - L 관계)을 합니다.
  - 카운팅 설정
    측정할 L 수, 타임아웃을 설정합니다.
  - 카운팅
    카운팅 설정 후 카운팅 동작이 수행됩니다.
    background counting으로 sleep 상태를 유지한 채로 counting만 수행합니다.
  - 종료
    카운팅 수가 목표치를 도달했거나, 타임아웃이 동작하면 MCU를 깨워 완료를 알립니다.

전원 버튼 드라이버

- 설명
  단일 버튼과 LED 하나로 기기의 상태를 제어하고 표현해야 합니다.

제어할 수 있는 기기 상태는 다음과 같습니다.
  - 활성화 상태
    기기가 현장에서 활성화된 상태입니다.
  - 비활성화 상태
    기기가 비활성화된 상태입니다. (배송, 겨울 때 보관)
    버튼으로만 킬 수 있습니다.
    BLE 로만 비활성화 상태로 진입할 수 있습니다.
  - BLE advertising 상태
    BLE advertising을 통해 주변 휴대폰 앱과 연결을 시도하는 상태입니다.
- 버튼 입력

패턴	동작 명칭	상세 설명
짧게 1번	상태 확인	현재 기기의 상태를 LED로 N초간 표시
3초간 길게	BLE 페어링	사용자와 앱 연결을 위해 BLE Advertising 시작
10초간 길게	기기 활성화	기기가 비활성화 상태라면 활성화 상태로 변경합니다.

- LED 상태 표시

패턴	상태 명칭	표시 조건	상세 설명
1박자 점멸	활성화 상태-정상	상태 확인 버튼 입력	딱, 딱 이렇게 5초간 표시
2박자 점멸	활성화 상태-저전력	상태 확인 버튼 입력	따딱, 따딱 이렇게 5초간 표시
항상 켜져 있음	BLE 페어링 상태	BLE 페어링 버튼 입력	BLE Advertising 상태 및 페어링 상태
3박자 점멸	기기 활성화	기기 활성화 버튼 입력	비활성화 -> 활성화 변경시에만 표시

- 필요 동작
  - 패턴 해석
    버튼 입력 패턴을 해석하고 해석이 완료되면 결과를 출력합니다.
  - 상태 표시
    상태를 표시하는 LED를 제어합니다.

도난 방지부 드라이버

- 설명
  진동 센서로 도난을 인식하고 도난이라고 판단되었을 시 부저를 울려 경고하며 LoRa 송신을 수행합니다.
- 필요 동작
  - 초기화
  - 진동 센서 패턴 해석
    진동 센서의 패턴을 해석해 도난이라고 판단되면 알림을 울립니다.
    예를 들어 특정 시간 동안 펄스가 n번 이상 발생하면 도난 시도로 판단합니다.
  - 도난 경보 활성화
    도난 경보를 활성화하여 부저를 울리며 정의된 동작이 수행하게 합니다.
  - 도난 경보 비활성화
    LoRa나 BLE로 도난 경보를 비활성화 합니다.

스케줄 기반 동작

스케줄 기반 동작에서는 오로지 절대 시점 기반의 인터럽트로만 동작하는 동작들을 정의합니다.

- 스케줄 예

    - 1번만
      2026/03/06/10:00
    - 일일 반복
      매일 10:00
    - 분단위 반복
      매일 00, 10, 20, 30, 40, 50분이 될 때 동작
- 해당하지 않는 예시 (인터럽트 기반 동작에서 다룸)
    - 10초 후
    - 버튼 클릭시

스케줄러 프레임워크

TaskScheduler나 RTOS의 다른 라이브러리등을 사용해 구현합니다.
스케줄러 프레임워크는 각 스케줄의 동작이 아닌, 스케줄의 등록, 해제, trigger를 지원함으로써 config나 외부 명령을 통해 동적으로 등록, 해제되는 기능을 지원하는 것이 중심입니다.
(이 이후부턴 TaskScheduler 중심으로 설명하지만, 다른 라이브러리도 충분히 동일하게 구현 가능)
스케줄은 task와 cron이 있습니다.

- task
  특정 시점에만 1회 동작하는 스케줄
  모든 명령이 이에 해당합니다.
  ex) 2026/03/06/10:00 에 관수 시작
- cron
  매일 또는 매분 반복해야 하는 스케줄
  최소 정밀도는 초 입니다.
  기본 cron은 무조건 지정된 cron_id를 점유합니다.
  이를 켜고 끄는 것은 enable로 지정 가능합니다.
  ex) 매일 10:00에 촬영
  ex) 00, 10, 20, 30, 40, 50분이 되면 센서 리딩
- task 인자

인자명	타입 (변경 가능)	설명	예시
task_id	uint16_t	스케줄 관리, 해제를 위한 지정에 필요한 값	1000
start_at	uint32_t	UNIX Timestamp 기반	1772955012
type	uint8_t	callback 함수 인덱스 (mapping등 사용)	
args	`uint8_t []`	callback 인자에 해당하는 데이터 바이트 배열 (callback 내부에서 구조체로 캐스팅해 사용)	
status	uint8_t	PENDING: 등록된 상태 RUNNING: 실행 중인 상태 COMPLETED: 완료된 상태 FAILED: 동작 중 실패한 상태 CANCELED: 외부 명령으로 취소된 상태 EXPIRED: 방전으로 인해 실행 중 방전됐거나 실행 조차 못한채로 상태가 지난 상태 REPORTED: 상태 보고가 완료된 상태 (이후 삭제됨)	PENDING=1
report	`uint8_t []`	실행 결과 리스트	

- cron 인자

인자명	타입 (변경 가능)	설명	예시
cron_id	uint16_t	스케줄 관리, 해제를 위한 지정에 필요한 값	2000
type	uint8_t	callback 함수 인덱스 (mapping등 사용)	
args	`uint8_t []`	callback 인자에 해당하는 데이터 바이트 배열 (callback 내부에서 구조체로 캐스팅해 사용)	
hour_mask	uint32_t	동작하는 hour를 기록하는 24비트 마스크	
min_mask	uint64_t	동작하는 min을 기록하는 60비트 마스크	
sec_mask	uint64_t	동작하는 sec을 기록하는 60비트 마스크	
last_run_at	uint32_t	마지막으로 실행된 시점의 Unix Timestamp 같은 '초'에 너무 빨리 돌아 중복 실행되는 것을 막는 용도	
enable	bool	True: 동작 False: 동작 안함	

- task 또는 cron 등록 방법
  스케줄러 프레임워크는 config에 등록된 인자를 바탕으로 이를 해석해 실제 task에 등록 및 동작시켜야 합니다.
  이를 위해 다음 과정을 거칠 것입니다.
  1. type -> callback 함수로 해석
     type-callback 맵핑 데이터를 바탕으로 대응되는 callback 함수 포인터를 가져옵니다.
  2. start_at, cron 해석
     UNIX 시간을 바탕으로 실제 delay time을 해석합니다.
  3. Task 구조체 생성 및 등록
     delay time, callback, args를 바탕으로 Task 구조체 생성 및 등록 (status = PENDING)
  4. Task 구조체 실행
     해당 time이 도래해 실행 (status = RUNNING)
  5. Task 구조체 완료
     status = COMPLETED
  6. 상태 발송
     명령 수행 상태를 송신하고 송신함을 확인함.
     status = REPORTED
  7. 정리
     나중에 config cleaner이 REPORTED로 된 config들을 삭제함.
- 스케줄러 프레임워크의 필요 동작
  - 초기화
    서버에서 시간을 받아와서 자체 UNIX 시간을 설정합니다.
    부팅되면서 config에 있는 인자들을 읽어서 등록합니다.
    누락된 스케줄이 있다면 이를 EXPIRED로 설정합니다.
  - config 등록
    수신한 config 인자를 config에 작성합니다.
  - config 갱신
    지정된 task_id, cron_id에 대해 수신한 config 인자를 config에 덮어씌웁니다.
  - 스케줄 등록
    위 config 인자를 받아 등록합니다.
  - 스케줄 갱신
    지정된 task_id, cron_id에 대해 등록된 스케줄을 취소하고 받은 인자 데이터로 스케줄을 재생성합니다.

- 스케줄 삭제
  지정된 task_id에 대해 스케줄 등록을 취소시키고 status = CANCLED로 전환합니다.
- 활성화/비활성화
  지정된 cron_id에 대해 enable을 true/false로 변경하고 스케줄 등록을 등록/취소 합니다.
- 스케줄 즉시 시작
  지정된 task_id에 대해 등록된 스케줄을 취소하고 해당 task_id를 즉시 시작합니다.

관수 벨브 ON/OFF 제어 Task

인자명	값
task_id	받은 command_id 값
type	MOTOR_VALVE_OPEN_CLOSE

- args
  - open/close
    개폐 지정값
  - 밸브_id
    밸브 번호 (0, 1 있음)
  - duration_sec (2바이트)
    open일 때만 저 duration 이전에 close 신호가 안들어왔을시 자동으로 close 함.
  - start_task_id
    실행하기 전에 실행할 task_id
  - end_task_id
    완료된 후에 실행할 task_id
- `MOTOR_VALVE_OPEN_CLOSE`
  (자신 status == RUNNING)
  args 구조체로 파싱
  start_task_id 있다면 실행
  해당 3-Line 모터 제어 드라이버에 개방/폐쇄 함수 사용
  duration 이후에 자동 close하는 동작은 3-Line 모터 제어 드라이버에 이미 구현됨
  end_task_id 실행
  성공시 status == COMPLETED
  성공한 task_id LoRa로 송신 시도
  송신 성공시 status == REPORTED

유량계 복합 제어 Task

인자명	값
task_id	받은 command_id 값
type	FLOWMETER_BASED_CONTROL

- args
  - 유량계_id
    유량계 번호 (0, 1 있음)
  - liter (2바이트)
    통과시킬 물 양
  - timeout (2바이트)
    timeout까지 liter이 도달 못하면 failed로 기록할 시점

    - start_task_id
      기록 시작할 때 trigger할 task_id ex) 밸브 열기
    - end_task_id
      폐쇄할 때 trigger할 task_id
      ex) 모터 정지, 밸브 닫기, 유량계 새로 liter 측정 등록
- 설명
  유량계는 그 스스로는 동작하지 않지만 동작 이전 이후의 task_id를 등록함으로써 다른 task_id를 제어하는 조건부 task가 됩니다.
  예를 들어 밸브 열어서 100L를 주는 임무에 대해서
  유량계 task에 100L를 넣고 start_task에 밸브 open, end_task에 밸브 close를 하고 유량계 task를 실행시키면
  (start) 밸브 열고 -> (task) 100L 측정하고 -> (end) 밸브 닫는 동작을 수행할 수 있습니다.
  주의할 점은 유량계 task는 스스로의 task 마저 귀납적으로 실행시킬 수 있으니 염두해 두어야 합니다.
- `FLOWMETER_BASED_CONTROL`
  (자신 status == RUNNING)
  args 구조체로 파싱
  종료시 trigger될 callback 함수 생성
  (FLOWMETER_BASED_CONTROL_END)
  유량계 드라이버를 통해 counting 활성화 및 (FLOWMETER_BASED_CONTROL_END) 함수 등록
  start_task_id 실행
- `FLOWMETER_BASED_CONTROL_END`
  종료 도달 조건 확인
  만약 timeout으로 도달했다면 status == FAILED (그래도 이 이후 진행)
  유량계 드라이버 해제
  next_task_id 실행
  status == COMPLETED
  성공한 task_id LoRa로 송신 시도
  송신 성공시 status == REPORTED

LoRa 송신 Task

인자명	값
task_id	받은 command_id 값
type	LORA_SEND

- args
    - data
      보내야할 data
- 설명
  특정 조건 도달시 (ex: 유량계) 다른 리비타 링크를 동작시켜야 할 때 필요한 Task입니다.
  다만 리비타 링크는 오로지 리비타 타워와만 통신할 수 있으므로 보내야할 ID는 항상 리비타 타워입니다.
- `LORA_SEND`
  (자신 status == RUNNING)
  args 구조체로 파싱
  LoRA로 data 송신
  status == COMPLETED
  성공한 task_id LoRa로 송신 시도
  송신 성공시 status == REPORTED

부저 알림 Task

인자명	값
task_id	받은 command_id 값
type	BUZZER_RUN

- 설명
  농민이 리비타 링크를 찾고 싶을 때 사용하는 기능입니다.
  1분간 울리며 농민이 찾기 쉽게 해줍니다.
- `BUZZER_RUN`
  (자신 status == RUNNING)
  args 구조체로 파싱
  도난 방지부 드라이버로 도난 경보 활성화 (1분간)
  status == COMPLETED
  성공한 task_id LoRa로 송신 시도
  송신 성공시 status == REPORTED

RS485 센서 리딩 Task

인자명	값
task_id	받은 command_id 값
type	RS485_SENSOR_READ

- 설명
  cron과 동일하지만 특정 시점에 읽고 싶을 때 동작시킬 수 있는 Task 입니다.
- args
  - `idx []`
    읽을 device(가상 드라이버)의 인덱스입니다.
    <보완 필요>

RS485 센서 리딩 Cron

인자명	값
cron_id	RS485_SENSOR
type	RS485_SENSOR_READ

- 설명
  N분 단위로 읽는, 리비타 링크의 핵심 스케줄입니다.
- args
  - `idx []`
    읽을 device(가상 드라이버)의 인덱스입니다.
    <보완 필요>

- `RS485_SENSOR_READ`
  args 구조체로 파싱
  RS485 드라이버 통해 RS485 드라이버 wakeup
  idx에 따라 가상 드라이버 1개씩 가동해 센서값 리딩
  센서 값 리딩 후 배치 형식으로 저장
  모두 읽기 완료했으면 LoRa 통해 배치로 전송
  전송 성공했으면 저장 자료 삭제
  성공 안했으면 저장 자료 보관 및 재시도

기기 상태 확인 Cron

인자명	값
cron_id	SELF_CHECK
type	SELF_CHECK_READ

- 설명
  N분 단위로 배터리 상태, 기기 온도, RAM 사용량 등을 체크합니다.
- `SELF_CHECK_READ`
  배터리 전압, 기기 온도, 로그 등 관리에 필요한 정보를 수집합니다.
  모두 수집했으면 LoRa 통해 배치로 전송
  전송 성공했으면 저장 자료 삭제
  성공 안했으면 저장 자료 보관 및 재시도

인터럽트 기반 동작

인터럽트 기반 동작에서는 모든 인터럽트가 아닌 새로운 동작을 발생시키는 인터럽트만 작성합니다. (timeout, duration 지정해서 동작하는 인터럽트는 여기선 언급 안함)
중복으로 인터럽트가 들어오면서 동작이 방해받아서는 안됩니다.

전원 버튼 입력 인터럽트

- 트리거 핀: BTN
- 트리거 조건: falling edge
- 설명
  버튼이 눌리면 sleep에서 깨어나서 전원 버튼 드라이버가 관련 동작을 수행합니다.

진동 센서 인터럽트

- 트리거 핀: VIB
- 트리거 조건: falling edge
- 설명
  진동을 감지해 도난 방지부 드라이버에 전송하여 도난을 감지합니다.

LoRa 수신 인터럽트

- 트리거 핀: 내부 DIO1
- 설명
  LoRa 패킷을 수신합니다, LoRa 드라이버가 이를 해석하여 관련 동작을 수행합니다.
- 관련 동작
  - 스케줄 Task 등록, 취소
  - 스케줄 Cron 등록, 취소
  - config 변경

    - OTA

리비타 타워

- 설명
  리비타 타워는 PTZ 카메라, LTE, 프로세서가 결합한 노지 스마트팜 관제 장치입니다.
  LTE를 통해 서버와 통신하며
  LoRa를 통해 리비타 링크에서 센서 데이터 수집 및 관수 장치 동작을 하며
  PTZ 카메라 및 프로세서를 동작해 현장을 촬영하고
  RS485 센서를 통해 조도 데이터를 수집합니다.
- 통신 관련 역할
  - LTE로 서버와 연결
    MQTT, 인증서 필요
  - LoRa로 리비타 링크의 master, gateway 역할 수행
    LoRa Mesh 오픈소스 사용
  - RS485로 기초 센서 데이터 수집
    RS485는 UART_MUX로 프로세서와 공유
  - BLE로 사용자의 앱에서 OTA, 등록 수행
- 관측 동작
  - PTZ 카메라를 통한 필지 스켄
    프로세스 구동 및 제어권을 프로세스에 양도해 프로세스가 PTZ 카메라 동작 및 LTE 업로드 (USB)
  - 해충 패드 예찰
    프로세스 구동 및 제어권을 프로세스에 양도해 프로세스가 해충 패드 촬영 및 LTE 업로드 (USB)
  - 기초 센서 데이터 수집
    RS485를 통해 조도 센서 데이터 수집
  - 리비타 링크 데이터 수신
    LoRa를 통해 수신된 데이터를 LTE로 서버에 업로드
- 제어 동작
  - 서버 명령 통한 PTZ 카메라 구동
    서버 요청 수신 시 프로세스 구동 및 제어권을 프로세스에 양도해 PTZ 카메라 동작
  - 서버 명령을 리비타 링크에 하달
    서버 명령을 LoRa를 통해 리비타 링크에 하달 및 하달 상태 확인
- 방범 동작
  - 도난 감지
    진동 센서를 통해 일정량 이상의 진동 발생시 부저음을 울리고 LTE로 도난 사실 송신
- 관리 동작
  - 전원 버튼
    전원 버튼을 특정 패턴으로 눌러서 기능 활성화
  - BLE 접속
    BLE를 접속해 사용자 앱에서 상태 확인 및 세부 설정 (ex: LTE 연결 없을 시 원격으로 상태 확인)
  - 상태 확인
    전원 버튼 옆의 LED의 점멸 패턴을 통해 기초 상태 표시 (ON, OFF, 페어링 등)

상세 동작 설명 (리비타 링크에서 추가되는 것, 변경되는 것만 설명)

HAL 레벨 동작

LoRa 드라이버, BLE 드라이버, Flash 드라이버, RS485 드라이버, 전원 버튼 드라이버

- HAL 레벨에선 리비타 링크와 동일=

3-Line 모터 제어 드라이버, 유량계 드라이버

- 제거

LTE + GPS 드라이버

- 설명
  리비타 타워에는 추가적으로 UART로 통신하는 LTE가 있습니다.
  LTE를 통해 서버와 통신을 수행하게 됩니다.
  리비타 타워 -> 서버로 송신하는 업링크 뿐만 아니라
  서버 -> 타워로 수신되는 다운링크 또한 대응이 가능해야 합니다. (인터럽트 기반)
  동일한 칩에 LTE 뿐만 아니라 GPS 또한 있으며 통신은 동일한 uart를 사용합니다.
- 필요 지원 동작
  - 초기화
  - 네트워크 구성
    인터넷 연결을 활성화합니다.
    이 동작은 필요시 무한반복하는 fallback이 있어야 합니다.
    인터넷 연결은 MQTT 기반으로 연결됩니다.
    이 시점에 무조건 시간 동기화가 되어야 합니다.
    (MQTT 인증서 등 지원 필수)
  - MQTT 등록
    인자로 받은 MQTT topic을 등록할 수 있어야 합니다.
  - 송신
    MQTT topic 형태로 데이터를 송신할 수 있어야 합니다.
  - 수신
    수신은 LTE RI 인터럽트와 연계되어 동작합니다.
    인터럽트로 wakeup 및 수신 데이터 처리를 지원합니다.
  - 진단
    원격 유지보수를 위해 RSSI, RSRP, RSRQ등의 상태 확인을 지원합니다.
  - 위치 정보 획득
    요청시 N분간 위치 정보를 획득하여 리턴합니다.

스케줄 기반 동작

프로세서 wakeup Task

인자명	값
task_id	받은 command_id 값
type	PROCESSOR_WAKE

- args
  - type
    프로세서가 동작 후 받아서 수행할 임무 type
- 설명
  대부분의 동작 (카메라 촬영, 병해충 패드 촬영)은 프로세서가 부팅되어서 동작하게 됩니다.
  MCU는 이를 위해 프로세서를 부팅하고, 실행해야할 임무가 담긴 data를 넘겨주는 방식으로 운영되게 됩니다.
  프로세서가 모든 임무를 마치면 프로세서 스스로 LTE에 데이터를 전송하고 종료됩니다.

- `PROCESSOR_WAKE`
  (자신 status == RUNNING)
  args 구조체로 파싱
  프로세서 전원 인가 및 UART 연결 시도
  프로세서 부팅이 완료되서 UART 연결이 되면 data를 프로세서에 전송
  프로세서는 제어권을 넘겨받고 받은 data 바탕으로 동작 수행
  동작 완료 후 MCU에 다시 제어권 이양
  MCU는 프로세서 전원 해제
  status == COMPLETED
  LTE로 task 송신 시도
  송신 성공시 status == REPORTED

프로세서 wakeup Cron

인자명	값
task_id	받은 command_id 값
type	PROCESSOR_WAKE

- args
  - type
    프로세서가 동작 후 받아서 수행할 임무 type
- 설명
  위 Task와 동일하나 Cron으로 동작합니다.

RS485 센서 리딩 Task, RS485 센서 리딩 Cron, 기기 상태 확인 Cron

리비타 링크과 공통이지만 송신이 LTE 송신으로만 변경됨

인터럽트 기반 동작

전원 버튼 입력 인터럽트, 진동 센서 인터럽트

리비타 링크와 동일

LoRa 수신 인터럽트

- 트리거 핀: 내부 DIO1
- 설명
  LoRa 패킷을 수신합니다, LoRa 드라이버가 이를 해석하여 관련 동작을 수행합니다. 리비타 링크와 달리 gateway로써 동작해줘야 합니다.
- 관련 동작
  - 리비타 링크의 스케줄 상태 송신을 LTE로 송신하기 (중계)
  - 리비타 링크의 LoRa 송신 Task 수행
  - LTE 미연결시 중계 데이터 보관

LTE 수신 인터럽트

- 트리거 핀: LTE_RI
- 설명
  LTE 데이터를 상시 대기하는 것이 아니라 LTE에 다운링크가 발생 시 LTE_RI의 인터럽트에 반응하여 UART로 해당 데이터를 가져오게 됩니다.
- 관련 동작

- 서버 명령 수신 (Task, Cron 등록, 취소)
- 서버 명령 중계
- config 변경
- OTA